«Код Будущего — программирование на Python для специалистов по информационной безопасности»

Модуль 2. Файлы, данные, простая обработка и визуализация.

ТЕМА Работа с датами и временем (datetime). Модули os / sys, безопасная работа с файловой системой. Описательная статистика

Практическое занятие

Цель:

Научиться работать с форматами времени и обращаться к командами системы.

Задачи:

Работа с датами и временем (datetime)

Модули os и sys

Безопасная работа с файловой системой

Описательная статистика в Python.

Методические указания и инструкции по выполнению задания

1. **Вспомните и повторите теорию, необходимую для выполнения практического задания**

Модуль datetime:

Классы: date, time, datetime, timedelta.

Операции: вычисление разницы дат, форматирование (strftime).

Применение: логирование, проверка сроков действия сертификатов.

Модуль os:

Управление файловой системой: os.listdir(), os.rename(), os.makedirs().

Безопасность: проверка существования файлов (os.path.exists), обработка путей.

Модуль sys:

Доступ к аргументам командной строки (sys.argv).

Завершение программы (sys.exit).

Описательная статистика:

Модуль statistics: среднее, медиана, дисперсия.

Пример: анализ времени отклика системы.

2. **Разберитесь в том, как выполняется задание**

Пример программы для работы с датами и временем.

Задание 1

Сравнить даты и найти разницу

Описание для тренажера:

1. Реализовать функцию для вычисления разницы между двумя датами.

2. Функция принимает два параметра `date1` и `date2` в виде строк формата "YYYY-MM-DD".

3. Преобразовать строки в объекты `datetime` с помощью `datetime.strptime`.

4. Вычислить абсолютную разницу в днях между датами (`abs((d2 - d1).days)`).

5. Вернуть число дней как результат функции.

6. Если формат даты неверный, вывести сообщение: Неправильный формат даты. Используйте 'YYYY-MM-DD'.

7. В случае ошибки формата возвращать `None`.

8. Не использовать запрещённые конструкции и модули: os, sys, subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20801-date-comparison

Решение:

```
from datetime import datetime

def date_difference(date1: str, date2: str) -> int:
```

```
try:
    d1 = datetime.strptime(date1, "%Y-%m-%d")
    d2 = datetime.strptime(date2, "%Y-%m-%d")
    return abs((d2 - d1).days)
except ValueError:
    print("Неправильный формат даты. Используйте 'YYYY-MM-DD'.")
    return None
```

Задание 2

Создать папку и проверить существование файла

Описание для тренажера:

1. Создать функцию, которая принимает два параметра: путь к папке `folder_path` и имя файла `file_name`.
2. Проверить, существует ли папка, если нет — создать её с помощью `os.makedirs` (`exist_ok=True`).
3. Если создание папки не удалось, вывести сообщение об ошибке и вернуть `False`.
4. Проверить существование файла в этой папке с помощью `os.path.isfile`.
5. Вывести сообщение о том, существует ли файл: `"Файл '{file_name}' существует/не найден в папке '{folder_path}'"`.
6. Вернуть `True`, если файл существует, иначе `False`.

7. Не использовать запрещённые конструкции и модули: subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20802-checking-file-existence

Решение:

```
import os

def create_folder_and_check_file(folder_path: str, file_name: str) -> bool:

    try:

        os.makedirs(folder_path, exist_ok=True)  # создаем папку

    except Exception as e:

        print(f"Не удалось создать папку: {e}")

        return False


    file_path = os.path.join(folder_path, file_name)

    exists = os.path.isfile(file_path)

    print(f"Файл '{file_name}' {'существует' if exists else 'не найден'} в папке '{folder_path}'")

    return exists
```

Задание 3

Найти в логах события за последние 5 минут

Описание для тренажера

1. Создать функцию, которая принимает путь к CSV-файлу логов `input_file` и параметр `minutes` (по умолчанию 5).

2. Загрузить CSV с помощью `pandas.read_csv`. Если файла нет

— вывести сообщение и вернуть пустой DataFrame.

3. Проверить, что в файле есть колонка `timestamp`. Если нет — вывести сообщение и вернуть пустой DataFrame.

4. Преобразовать колонку `timestamp` в datetime с помощью `pd.to_datetime`. В случае ошибки — вывести сообщение и вернуть пустой DataFrame.

5. Определить текущий момент времени (`datetime.now()`) и вычислить отсечку `cutoff = now - timedelta(minutes=minutes)`.

6. Отфильтровать строки, где `timestamp >= cutoff`.

7. Вывести количество найденных событий за последние N минут.

8. Вернуть DataFrame с найденными событиями.

9. Не использовать запрещённые конструкции: subprocess, socket, importlib, builtins, eval, exec, compile.

Тренажер - 20803-events-in-logs

Решение:

```
import pandas as pd

from datetime import datetime, timedelta

def get_recent_logs(input_file="logs.csv", minutes=5):

    try:

        df = pd.read_csv(input_file)

    except FileNotFoundError:

        print(f"Файл {input_file} не найден")
```

```
        return pd.DataFrame()  # возвращаем пустой DataFrame

    if "timestamp" not in df.columns:
        print("Файл должен содержать колонку 'timestamp'")
        return pd.DataFrame()

    try:
        df["timestamp"] = pd.to_datetime(df["timestamp"])
    except Exception:
        print("Не удалось преобразовать 'timestamp' в формат даты/времени")
        return pd.DataFrame()

    now = datetime.now()
    cutoff = now - timedelta(minutes=minutes)
    recent_df = df[df["timestamp"] >= cutoff]
    print(f"Найдено {len(recent_df)} событий за последние {minutes} минут")
    return recent_df
```

Задание 4

Прочитать лог-файл

Пользователь вводит временной диапазон (start и end)

Найти события ERROR в этом интервале

Сохранить результат в filtered_log.txt

Описание для тренажера:

1. Запросить у пользователя начало и конец временного диапазона в формате `ГГГГ-ММ-ДД ЧЧ:ММ:СС`.

2. Преобразовать введённые строки в объекты `datetime`. Если формат неверный — вывести ошибку.

3. Открыть исходный лог-файл для чтения и файл `filtered_log.txt` для записи.

4. Для каждой строки:

   - Игнорировать, если нет слова `"ERROR"`.

   - Попытаться извлечь временную метку из начала строки (первые два элемента, `YYYY-MM-DD HH:MM:SS`).

   - Преобразовать её в `datetime`.

   - Если временная метка попадает в диапазон `[start_time, end_time]`, записать строку в выходной файл.

5. В конце вывести сообщение о сохранении результатов.

6. Обрабатывать ошибки: файл не найден, неверный формат даты, ошибки при разборе строк.

Важно: запрещённые конструкции вроде `subprocess`, `eval`, `exec` использовать нельзя (только стандартное чтение файлов и `datetime`)

Тренажер - 20804-read-log-file

Пример корректного решения:

```
from datetime import datetime
def filter_logs_by_time(log_file="app.log", output_file="filtered_log.txt"):
    Фильтрует логи по временному диапазону
    try:
        start = input("Введите начало периода (ГГГГ-ММ-ДД ЧЧ:ММ:СС): ")
        end = input("Введите конец периода (ГГГГ-ММ-ДД ЧЧ:ММ:СС): ")

        start_time = datetime.strptime(start, "%Y-%m-%d %H:%M:%S")
        end_time = datetime.strptime(end, "%Y-%m-%d %H:%M:%S")

        with open(log_file, 'r') as infile, open(output_file, 'w') as outfile:
            for line in infile:
                if "ERROR" not in line:
                    continue

                try:
                    log_time_str = line.split()[0] + ' ' + line.split()[1]
                    log_time = datetime.strptime(log_time_str, "%Y-%m-%d
%H:%M:%S")

                    if start_time <= log_time <= end_time:
                        outfile.write(line)
                except (IndexError, ValueError):
                    continue
```

```
        print(f"Отфильтрованные логи сохранены в {output_file}")

    except FileNotFoundError:
        print(f"Файл {log_file} не найден")
    except ValueError:
        print("Неверный формат даты. Используйте ГГГГ-ММ-ДД ЧЧ:ММ:СС")
# Пример использования
filter_logs_by_time()
```

Задание 5

Реализовать анализ логов:

Подсчёт количества ошибок

Среднее время между ошибками

Самое раннее и позднее событие

Сформировать краткий отчёт в файл (txt или csv)

Описание для тренажера:

Прочитать лог-файл (`app.log`) и собрать все строки с `"ERROR"`.

Из каждой строки извлечь временную метку (`YYYY-MM-DD HH:MM:SS`) и преобразовать в `datetime`.

На основе этих временных меток:

- Подсчитать количество ошибок.
- Найти самое раннее и самое позднее событие.
- Посчитать среднее время между ошибками в секундах (если

ошибок больше одной).

Сформировать отчёт с результатами и сохранить его в текстовый файл (`report.txt` или другой, указанный пользователем).

Обрабатывать ошибки:

- Файл не найден.
- Невозможно преобразовать строку в дату.
- Ошибки при сохранении отчёта.

Тренажер - 20805-log-analysis

Решение:

```
from datetime import datetime

import csv

def analyze_logs(log_file="app.log", output_file="report.txt"):

    try:

        with open(log_file, "r", encoding="utf-8") as f:

            lines = f.readlines()

    except FileNotFoundError:

        print(f"Файл {log_file} не найден")

        return


    error_times = []
```

```
    for line in lines:
        if "ERROR" in line:
            try:
                timestamp_str = line.split()[0] + " " + line.split()[1]
                timestamp = datetime.strptime(timestamp_str, "%Y-%m-%d %H:%M:%S")
                error_times.append(timestamp)
            except (IndexError, ValueError):
                continue

    report_lines = []
    report_lines.append(f"Количество ошибок: {len(error_times)}")

    if error_times:
        error_times.sort()
        earliest = error_times[0]
        latest = error_times[-1]
        report_lines.append(f"Самое раннее событие: {earliest}")
        report_lines.append(f"Самое позднее событие: {latest}")

        if len(error_times) > 1:
            diffs = [(error_times[i] - error_times[i-1]).total_seconds() for i in range(1,
len(error_times))]
            avg_diff = sum(diffs) / len(diffs)
            report_lines.append(f"Среднее время между ошибками (сек): {avg_diff:.2f}")
        else:
            report_lines.append("Среднее время между ошибками: N/A")
    else:
        report_lines.append("Нет ошибок для анализа")
```

```
# Сохраняем отчёт в текстовый файл
try:
    with open(output_file, "w", encoding="utf-8") as f:
        for line in report_lines:
            f.write(line + "\n")
    print(f"Отчёт сохранён в {output_file}")
except Exception as e:
    print(f"Не удалось сохранить отчёт: {e}")
```

Критерии оценивания

Критерий	**Баллы**	**Требования**
1. Корректность работы программы	0-1	– 1 балл: программа работает без ошибок, полностью соответствует условиям задачи, и выводит правильный ответ. – 0 баллов: программа не работает или выдает критичные ошибки, выводит неправильный ответ.
2. Использование необходимых функций	0-2	– 2 балла: при решении использованы все требуемые необходимые языковые средства – 1 балл: при решении использованы не все необходимые языковые средства – 0 баллов: требуемые языковые средства не использованы

Шкала оценивания:

Зачтено — 3 балла

Не зачтено — 0 – 2 балла

Требования к оформлению результата выполнения задания: решение задачи предусматривает ввод программного кода в специальное поле тренажера.